

Stack Overflows

CS 465 · Introduction to Security and Privacy · Fall 2026

BYU



Stack Overflow: Still the Canonical Bug

Decades old

Described in the 1970s, weaponized in the 1980s, still being found in shipped C today.

Still everywhere

Search [a CVE database for "overflow"](#) and the list does not stop scrolling — kernels, parsers, firmware.

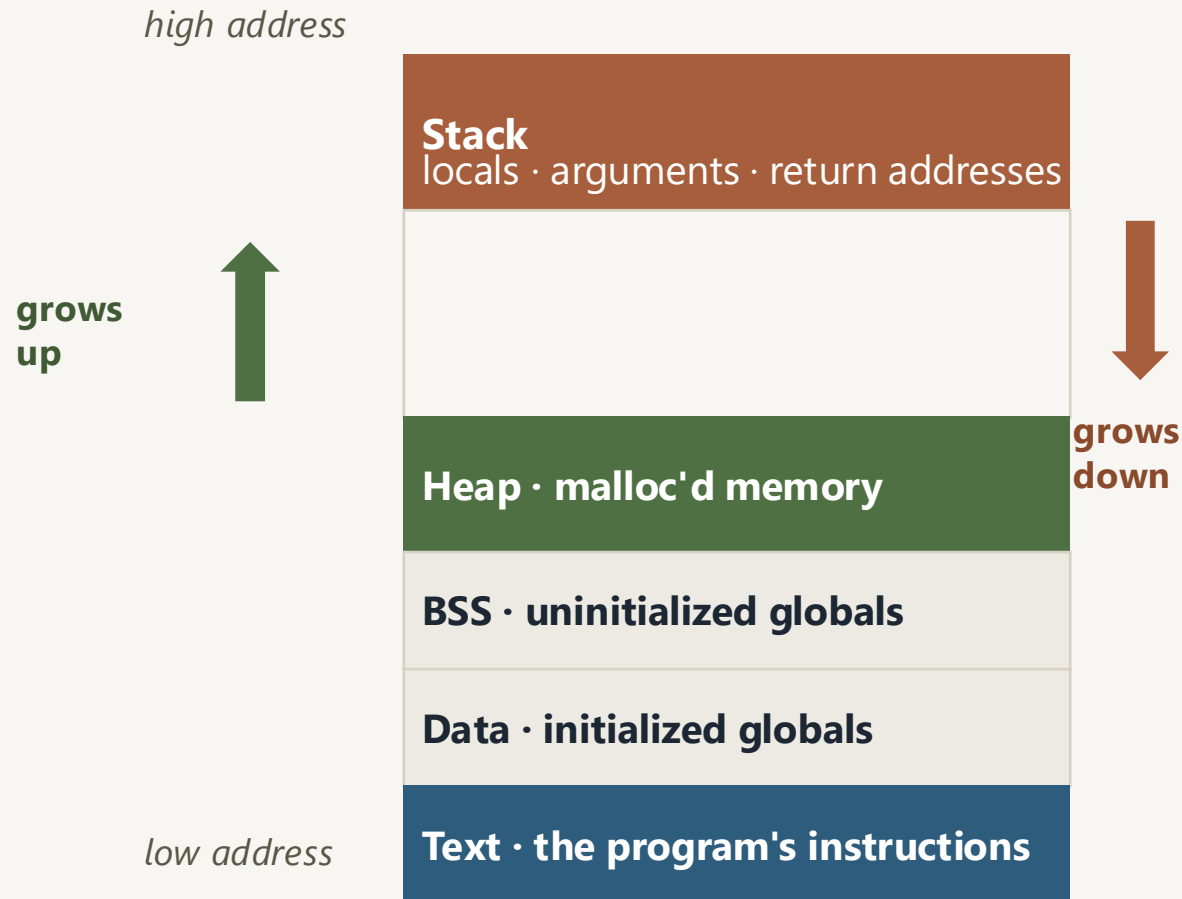
The reference attack

Heap overflows, format strings, ROP — all of them get explained by comparison to this one.

The recipe never changes

A fixed-size buffer, an unchecked length, and a copy that keeps going — and the unchecked length may arrive courtesy of the integer bugs we just finished.

How a Process Lays Out Memory



Why the stack

Heap and stack grow toward each other through the free space between them.

The stack is the interesting target: it holds the return addresses that decide where the program goes next.

The Goal

Write attacker-chosen code onto the stack — then get the program to execute it.

Problem 1 — get the bytes in

Overflow a stack buffer so attacker data lands in memory the program will run.

Problem 2 — get control there

Overwrite the saved return address so ret jumps into those bytes.

The Stack Frame



Why this shape matters

Addresses increase upward — and a buffer fills upward too.

The saved return address sits above the locals, so an overflowing local runs straight into it.

FP points at the saved old FP:
locals below, arguments above.

What Happens on a Call

Making the call

1. Caller pushes the arguments onto the stack.
2. call pushes IP as the return address, then jumps to the function.
3. Callee pushes the old FP so it can be restored later.
4. $FP \leftarrow SP$. Now $FP+k$ reaches args, $FP-k$ reaches locals.

Coming back

5. SP is decremented, carving out space for the locals.
6. The function body runs until it is ready to return.
7. Cleanup: $SP \leftarrow FP$, then $FP \leftarrow \text{old FP}$ popped off the stack.
8. ret pops the return address into IP — execution resumes in the caller.

Step 8 is the key to the attack: ret trusts whatever sits in the return-address slot.

A Function That Can Be Overflowed

```
void myfunction(char *src)    /* src is a ptr to a char string */
{ int var1, var2;            /* 1 stack word used per integer */
  char var3[4];              /* also 1 word for 4-byte buffer */

  strcpy(var3, src);          /* template: strcpy(dst, src) */
}
```

var3 holds three characters

Four bytes: three chars plus the NUL that C uses to mark end-of-string.

Word size is not the point

The example assumes 4-byte words; the attack works the same at any word size.

Why strcpy is a Problem

src — attacker-controlled: 9 characters, then NUL

A	A	A	A	A	A	A	A	A	\0
---	---	---	---	---	---	---	---	---	----

destination — var3[4], and everything the copy keeps going into

A	A	A	\0	var2 · var1 · old FP · return address					
---	---	---	----	---------------------------------------	--	--	--	--	--

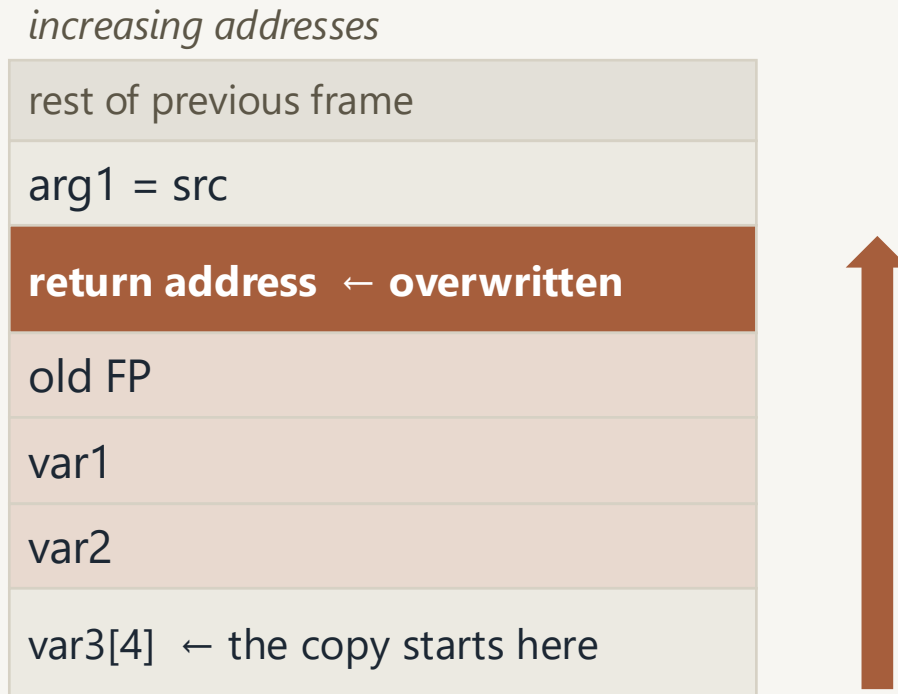
↑ *all var3 can hold*

↑ *written anyway*

strcpy never learns how big the destination is

It copies byte for byte and stops only after it copies a NUL from the source. The destination's size never enters the decision — and the unchecked length often arrives courtesy of the integer bugs from §6.2.

The Overflow, Step by Step



the copy marches upward through every word above var3

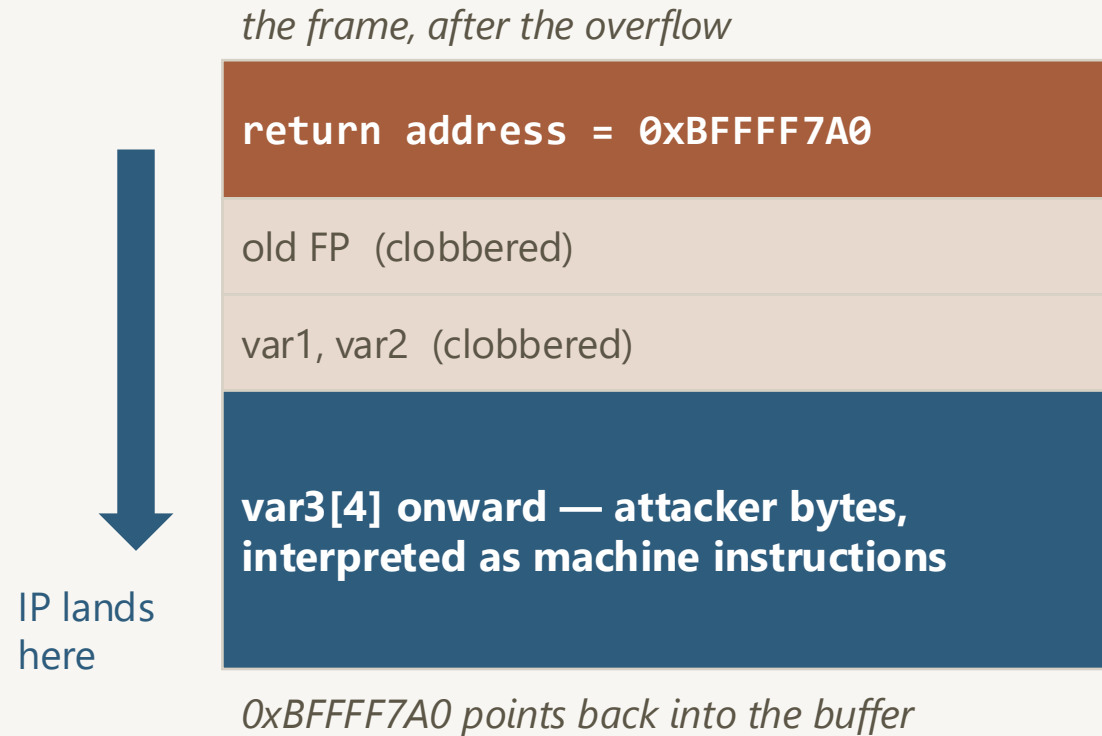
If $n > 3$, the spill begins

var3 holds three characters and a NUL. A longer src keeps writing into var2, then var1, then the saved old FP.

Keep going and the write reaches the return address — the one word that decides where the function goes next.

On return, IP is loaded from that slot.

Hijacking Control Flow



Input becomes code

Control still transfers — just to the overwriting value. Point it back into the buffer and the CPU fetches instructions from the stack.

If those bytes are valid opcodes, the machine runs the attacker's program.

Needed: valid instruction bytes, and an address that reaches them.

The No-Op Sled

the attacker cannot predict the landing address exactly — so widen the target



NOP NOP NOP NOP NOP NOP NOP NOP NOP NOP

injected code

return addr

land anywhere in here ...

... and you slide into this

Precision is the hard part

The attacker must predict where the injected code will sit, and put that address at exactly the offset that lands on the return-address slot.

A sled of NOPs buys slack: any landing inside it runs forward into the code.

Sleds Are a Tell

```
; x86 NOP = 0x90
```

```
90 90 90 90 90 90 90 90
```

```
; no-effect equivalents
```

```
or    eax, 0
```

```
add   eax, 0
```

A literal run is a signature

Long stretches of 0x90 are a telltale sign of an attack — exactly the pattern scanners and intrusion detection look for.

So attackers vary it

Swap in instructions that do nothing useful but are not NOPs. The sled still works; the signature disappears, and discovery gets much harder.

Demo — The Vulnerable Program

```
# github.com/BYU-CS465/software-security  
$ cd buffer-overflow  
$ ./stack
```

What it is

- The toy example from the book, made real.
- Same shape as myfunction: a fixed buffer and an unchecked copy.

What to watch

- Short input: behaves normally.
- Longer input: the frame is damaged and the program goes somewhere it was never asked to go.

Demo — exploit.py

```
# padding to reach the slot, then the address  
$ python3 exploit.py  
$ ./stack < payload
```

What the script does

- Builds an input long enough to reach the return-address slot.
- Writes a return address of our choosing into it.

Walkthrough

- See [stack overflow walkthrough](#)